EF Core: Distinct() + GroupBy()

Reports & real-world queries | Concise lesson summary

Core idea

Distinct() returns unique values by removing duplicates. GroupBy() groups rows that share a key so you can build summaries such as counts per status, courses per instructor, or enrollments per course.

Mental anchor: Distinct() removes repeats. GroupBy() turns rows into report groups.

When to use each method

Method	What it does	Typical use
Distinct()	Removes duplicate values from the selected result.	List of unique student statuses, countries, or course levels.
GroupBy()	Groups rows by a key; each group can be summarized.	Number of students per status, courses per instructor, revenue per category.

1. Distinct(): get unique values

UniqueStatuses.cs — C# • EF Core

```
1  var query = context.Students
2  .Select(s => s.Status)
3  .Distinct();
4
5  var statuses = query.ToList();
```

Select the column you want unique values for, then apply Distinct(). ToList() executes the query and materializes the small result list.

Typical SQL shape

Distinct.sql — C# • EF Core

```
1  SELECT DISTINCT [s].[Status]
2  FROM [Students] AS [s];
```

2. GroupBy(): build a report

StudentsPerStatus.cs C# • EF Core

```
var query = context.Students
.GroupBy(s => s.Status)
.Select(g => new
{
Status = g.Key,
TotalStudents = g.Count()
})
.OrderBy(x => x.Status);

var report = query.ToList();
```

g.Key is the value used to group rows. g.Count() counts the rows inside each group. The projection creates a report-shaped result.

Typical SQL shape

GroupBy.sql C# • EF Core

```
SELECT [s].[Status], COUNT(*) AS [TotalStudents]
FROM [Students] AS [s]
GROUP BY [s].[Status]
ORDER BY [s].[Status];
```

Avoid grouping in memory

AvoidThis.cs C# • EF Core

```
var students = context.Students.ToList();
var report = students.GroupBy(s => s.Status);

// Prefer database-side grouping for a report
var report = context.Students
.GroupBy(s => s.Status)
.Select(g => new { Status = g.Key, Count = g.Count() })
.ToList();
```

The first version loads all student rows before grouping. The second lets EF Core translate the report query to SQL when the grouping/projection is translatable.

Common mistake: Distinct() on a full entity

ChooseTheProjection.cs C# • EF Core

```
// Usually, when you need unique status values:
var statuses = context.Students
.Select(s => s.Status)
.Distinct()
.ToList();
```

Applying Distinct() to full entities is not the same as asking for the unique values of one column. Select the data the report actually needs.

SQL preview and runtime logging

ToQueryString() previews the SQL query shape for an IQueryable query; it does not execute it. **Runtime logging** configured through LogTo() shows SQL actually executed when the query is materialized or otherwise executed. Use both to understand translation and execution.

Practical checklist

- Use Select(...).Distinct() when you need unique values from a specific column.
- Use GroupBy(key).Select(...) to shape groups into useful report rows.
- Add aggregates such as Count() inside the grouped projection to summarize each group.
- Keep grouping and aggregation in the database when the query can be translated, especially for large tables.
- Use OrderBy() on the projected report if you need predictable display order.

Professional use cases

Dashboards • charts • admin reports • student status summaries • course/instructor summaries • enrollments per course

> **Remember: Distinct() answers “which unique values exist?” GroupBy() answers “how are rows distributed across groups?”**